ESPECIFICACIONES TÉCNICAS Y GUÍA DE INFRAESTRUCTURA DE LABORATORIO

REQUERIMIENTOS: CLÚSTER KUBERNETES MULTI-NODO

FACULTAD DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS INTELIGENTES

DEPARTAMENTO DE COMPUTACIÓN Y SISTEMAS

METADATOS DE CONFIGURACIÓN

ESTÁNDAR DOCENTE

CURSO:	Ingeniería de Software V	**CÓDIGO:**	09776 - TIC
PERÍODO:	2026-2 • NRC: 11005	**AMBIENTE:**	Laboratorio Presencial / Red IasLab
TAMAÑO GRUPO:	**7 Estudiantes por Equipo**	**MÁQUINAS:**	**7 Computadores Dedicados**
DISTRIBUCIÓN:	1 Control Plane + 5 Workers + 1 Bastion	**ENTORNO:**	**Docker-Native (rancher/k3s sin sudo)**

1 Visión General y Topología de 7 Computadores por Grupo

El presente documento define los requerimientos de hardware, sistema operativo, configuración de red, cortafuegos y distribución de roles para la ejecución del taller práctico y evaluativo de **Kubernetes Multi-Nodo**. La infraestructura está diseñada para que cada equipo de trabajo opere con **7 computadores físicos o instancias dedicadas**, garantizando que cada estudiante asuma la responsabilidad directa sobre una máquina y sus cargas de trabajo asociadas.

Arquitectura Docker-Native y Seguridad Institucional: Para proteger la integridad del sistema operativo en salas de cómputo compartidas, los estudiantes operan **exclusivamente con el usuario `docker` sin permisos de `sudo` ni elevación a root**. El clúster se despliega levantando la imagen oficial `rancher/k3s` como contenedor con las opciones `--privileged --net=host`, permitiendo enlazar la red física de los 7 computadores, aislar los datos y limpiar todo en segundos.

1.1 Matriz de Asignación y Topología de Red

Nodo	Hostname Oficial	IP Sugerida	Rol K8s / Cargas	Responsabilidad del Estudiante
PC 1	`k8s-control-plane`	`10.147.20.101`	Control Plane (Master)	Kube-apiserver, etcd, scheduler, control RBAC y token de unión.
PC 2	`k8s-worker-backend-1`	`10.147.20.102`	Worker Node 1	Ejecución de microservicios Backend A (Réplicas 1 y 2).
PC 3	`k8s-worker-backend-2`	`10.147.20.103`	Worker Node 2	Réplicas de Backend B (Alta disponibilidad y absorción de fallos).
PC 4	`k8s-worker-frontend`	`10.147.20.104`	Worker Node 3	Despliegue de Single Page Application / Nginx Frontend Web.
PC 5	`k8s-worker-data`	`10.147.20.105`	Worker Node 4	Base de Datos PostgreSQL, caché Redis y persistencia de disco.
PC 6	`k8s-worker-ingress`	`10.147.20.106`	Worker Node 5 (Edge)	NGINX Ingress Controller, balanceo L7 y terminación SSL.
PC 7	`k8s-bastion-ops`	`10.147.20.107`	Nodo Bastion / SRE	Generación de tráfico (`hey`), simulación de caos y auditoría.

2 Requerimientos de Hardware por Computador

Cada una de las 7 máquinas del equipo debe satisfacer los siguientes umbrales de recursos:

- **Procesador (CPU):** Mínimo 2 núcleos físicos o lógicos x86_64 (Recomendado: 4 núcleos para PC 1 Control Plane).
- **Memoria RAM:**
  - **PC 1 (Control Plane):** Mínimo 4 GB de RAM (para sostener `etcd` y `kube-apiserver` holgadamente).

- **PC 5 (Data / Database):** Mínimo 4 GB de RAM (para buffers de PostgreSQL y caché).
- **PC 2, 3, 4, 6 y 7 (Workers y Bastion):** Mínimo 2 GB de RAM (Recomendado: 4 GB).

- **Almacenamiento en Disco:** Mínimo 20 GB de espacio disponible en partición raíz (`/`) con almacenamiento SSD o HDD de alto desempeño.
- **Conectividad de Red:** Interfaz Ethernet Gigabit (1000BASE-T) o adaptador Wi-Fi de 5 GHz estable. Todas las 7 máquinas deben comunicarse entre sí sin restricciones de aislamiento en capa 2/capa 3 (subred compartida o red virtual ZeroTier asignada).

3 Requerimientos de Sistema Operativo y Entorno Docker-Native (Sin Sudo)

1. **Sistema Operativo Base:** Ubuntu Server / Desktop 22.04 ó 24.04 LTS con Docker Engine instalado y en ejecución.
2. **Privilegios de Usuario:** Acceso al grupo `docker` (`docker ps` funcional). **No se requieren permisos de `sudo` ni contraseña de root**.
3. **Desacople de Memoria Swap:** Al ejecutar K3s en contenedor Docker con la bandera `--kubelet-arg="fail-swap-on=false"`, **NO es necesario ejecutar `swapoff -a` ni modificar `/etc/fstab`** en el host.
4. **Enrutamiento de Red y Puentes:** El daemon de Docker gestiona de forma nativa los módulos del kernel (`overlay`, `br_netfilter`) y el parámetro `ip_forward=1` al inicializarse, eliminando la necesidad de comandos administrativos.
5. **Imagen Oficial de Kubernetes:** Imagen Docker certificada `rancher/k3s:v1.31.1-k3s1` descargada previamente o al momento del inicio (65 MB).
6. **Herramientas CLI:** En las máquinas del laboratorio: `docker`, `kubectl` (disponible en `/usr/bin/kubectl` o configurable en el PATH local del usuario) y `hey`.

4 Matriz de Puertos y Reglas de Firewall

Si las máquinas tienen activo el cortafuegos (`ufw` o `iptables`), deben autorizarse los siguientes puertos:

Protocolo / Servicio	Puerto	Tipo	Origen	Destino y Propósito
Kubernetes API Server	6443	TCP	Todo el clúster	PC 1 (Control Plane): Entrada de la API REST para `kubectl` y Kubelets.
etcd Server Client API	2379-2380	TCP	Control Plane	PC 1: Base de datos interna del clúster (solo localhost y planos de control).
Kubelet API	10250	TCP	Control Plane	Todos los Workers (PC 2 a PC 6): Comunicación del master con los agentes.
Flannel VXLAN Overlay	8472	UDP	Todo el clúster	Todos los nodos: Comunicación Pod-to-Pod encapsulada en red CNI.
Ingress HTTP / HTTPS	80 / 443	TCP	Clientes externos	PC 6 (Ingress): Entrada pública de tráfico web balanceado.
NodePort Services	30000-32767	TCP	Red LAN / Clientes	Todos los Workers: Rango institucional para servicios expuestos por NodePort.

Comando de apertura rápida en UFW (si está activo):

```
# En PC 1 (Control Plane)
sudo ufw allow 6443/tcp && sudo ufw allow 8472/udp && sudo ufw allow 10250/tcp

# En PC 2, 3, 4, 5, 6 (Workers)
sudo ufw allow 10250/tcp && sudo ufw allow 8472/udp && sudo ufw allow 30000:32767/tcp

# En PC 6 adicionalmente
sudo ufw allow 80/tcp && sudo ufw allow 443/tcp
```

5 Matriz Organizacional de Roles para el Equipo de 7 Estudiantes

Para garantizar un trabajo colaborativo ordenado y evitar cuellos de botella, cada uno de los 7 miembros asumirá un rol especializado dentro de la práctica:

PC	Rol Asignado	Estudiante Responsable	Responsabilidades Clave
1	**Control Plane Lead**	Estudiante 1	Inicializa el master, distribuye el token y `kubeconfig`, supervisa la unión de nodos y gestiona Namespaces y RBAC.
2	**Backend Core Engineer**	Estudiante 2	Configura Worker 1, despliega la versión base del backend, verifica requests/limits y pruebas de liveness.
3	**Backend HA & Scaling Lead**	Estudiante 3	Configura Worker 2, implementa las reglas de `podAntiAffinity`, monitorea la absorción de réplicas y rollouts.
4	**Frontend UI Engineer**	Estudiante 4	Configura Worker 3, compila el artefacto web, despliega el frontend y verifica su conectividad con el backend.
5	**Data & Storage Lead**	Estudiante 5	Configura Worker 4, gestiona volúmenes persistentes (`PV/PVC`), Secrets de base de datos y credenciales.
6	**Ingress & Gateway Admin**	Estudiante 6	Configura Worker 6, instala el Ingress Controller, define reglas de routing L7 por host/path y gestiona certificados.
7	**SRE, Chaos & QA Lead**	Estudiante 7	Opera desde el nodo Bastion, ejecuta pruebas de estrés con `hey`, coordina el corte de red y audita resiliencia.

6 Checklist de Aprobación de Laboratorio (Validación Docente)

Antes de finalizar la sesión presencial de 3 horas, el equipo debe presentar la siguiente lista de chequeo al docente o monitor:

☐ **Check 1: Clúster 100 % Saludable:** Ejecución de `kubectl get nodes -o wide` mostrando los 6 nodos del clúster en estado `Ready` con sus etiquetas semánticas asignadas.

☐ **Check 2: Cargas Distribuidas y Anti-Afinidad:** Ejecución de `kubectl get pods -n produccion -o wide` demostrando que los pods de backend están distribuidos en distintas máquinas físicas (PC 2 y PC 3) y la base de datos corre en PC 5.

☐ **Check 3: Enrutamiento L7 por Dominio:** Petición exitosa desde PC 7 mediante `curl -I http://ingesoft5.lab.icesi/api` y `http://ingesoft5.lab.icesi/` resolviendo a través del Ingress en PC 6.

☐ **Check 4: Resistencia ante Fallo Físico (Chaos Test):** Desconexión en vivo del cable de red de PC 2 durante una ráfaga de tráfico activa sin que el servicio se interrumpa ($\geq 99{,}5\,\%$ éxito).

☐ **Check 5: Rollout y Rollback Exitoso:** Demostración de actualización a versión v2 y reversión inmediata tras fallo provocado.